

Authentication JWT

Séance 6 - Sécuriser son backend : qui es-tu, et qu'as-tu le droit de faire ?

Cours 420-941-MA - Collège de Maisonneuve - Groupe 25604

sessions vs tokens - bcrypt - JWT - register/login - middleware - rôles (RBAC) - sécurité

Où on en est, où on va

Récap S5 et objectifs S6

Acquis (S1 -> S5)

- API REST avec Express (routes, middlewares)
- CRUD complet avec Prisma + Neon
- Consommer des APIs avec axios
- Tester avec Postman / REST Client
- MAIS : toutes nos routes sont OUVERTES

Objectifs de la séance

- Authentification vs autorisation
- Sessions vs tokens (JWT)
- Hacher les mots de passe (bcrypt)
- Émettre et vérifier un JWT
- Protéger des routes (middleware)
- Gérer les rôles (USER / ADMIN)

Le problème. Aujourd'hui, N'IMPORTE QUI peut appeler DELETE /pokedex/3 ou créer des comptes. Notre API ne sait pas QUI parle ni ce qu'il a le DROIT de faire. Une vraie application doit savoir reconnaître ses utilisateurs et protéger ses actions sensibles. C'est tout l'enjeu de cette séance.

PARTIE 1

Le besoin d'authentification

Deux notions à ne pas confondre

Authentication vs Autorisation

Authentication (authN)

- QUI es-tu ?
- Prouver son identité
- Ex : email + mot de passe
- Résultat : on sait qui tu es

Autorisation (authZ)

- As-tu le DROIT ?
- Vérifier les permissions
- Ex : seul un ADMIN peut supprimer
- Résultat : action permise ou refusée

Analogie : entrer dans un immeuble

- **Authentication** : montrer sa carte d'identité à l'entrée (prouver qui on est)
- **Autorisation** : votre badge ouvre le 3e étage, mais pas la salle des serveurs
- **On s'authentifie d'abord**, on est autorisé ensuite

Le défi : HTTP est sans mémoire

Chaque requête repart de zéro

HTTP est stateless (sans état) : le serveur ne se « souvient » pas de la requête précédente. Si vous vous connectez à la requête 1, le serveur l'a déjà oublié à la requête 2. Comment, alors, rester connecté ?

Le problème concret

- POST /login -> OK, c'est bien Alice
- GET /profil -> ... qui es-tu déjà ?
- Le serveur a tout oublié
- Renvoyer son mot de passe à CHAQUE
- requête ? Hors de question.

La solution

- Au login, le serveur remet une PREUVE
- Le client la renvoie à chaque requête
- Le serveur vérifie la preuve
- Deux familles : sessions ou tokens
- (on compare juste après)

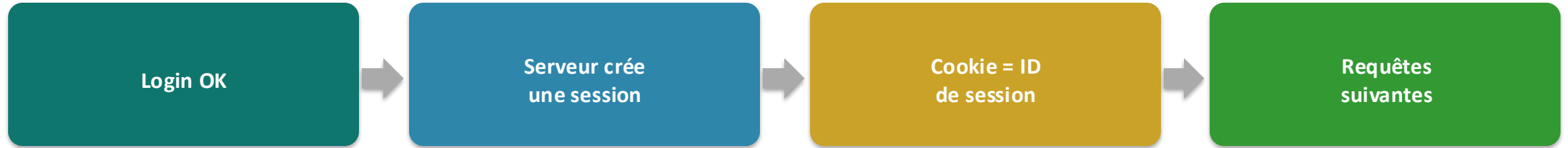
Cette « preuve » prend la forme d'un identifiant de session (stocké côté serveur) OU d'un token signé (stocké côté client). Les deux approches répondent à la même question : comment prouver, à chaque requête, qu'on s'est déjà connecté.

PARTIE 2

Sessions vs Tokens (JWT)

Approche 1 : sessions + cookies

L'état est gardé côté SERVEUR (stateful)



Avantages

- Révocation immédiate (suppr. la session)
- ID opaque : rien à lire dedans
- Mûr, bien outillé (express-session)

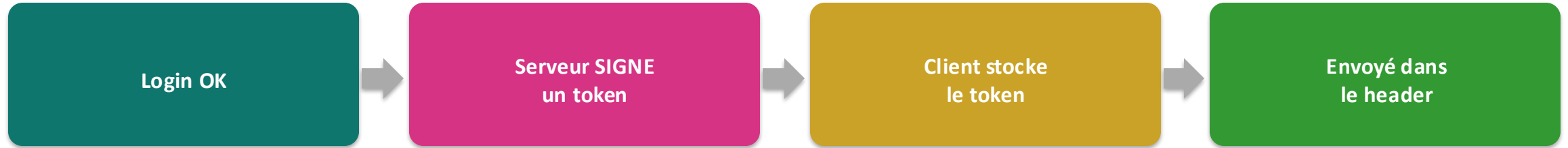
Inconvénients

- Le serveur doit STOCKER les sessions
- Difficile à répartir (plusieurs serveurs)
- Couplé aux cookies (CSRF à gérer)

Idée clé : avec les sessions, le client ne porte qu'un identifiant ; la VRAIE information (qui es-tu) vit dans la mémoire / la base du serveur.

Approche 2 : tokens (JWT)

L'état voyage dans le TOKEN (stateless)



Avantages

- Le serveur ne STOCKE rien (stateless)
- Idéal pour APIs / microservices / mobile
- Se vérifie avec juste un secret

Inconvénients

- Révocation difficile (valide jusqu'à exp.)
- Le payload est LISIBLE (pas de secrets !)
- Garder le token en sécurité côté client

C'est l'approche qu'on adopte en S6 : parfaite pour une API REST consommée par axios (et bientôt par un frontend React).

PARTIE 3

Les mots de passe : bcrypt

Règle d'or : jamais en clair

Un mot de passe ne se stocke pas, il se HACHE

Si votre base fuit et que les mots de passe sont en clair, c'est la catastrophe (et souvent les gens réutilisent le même partout). On ne stocke donc JAMAIS le mot de passe : on stocke son EMPREINTE (hash).

Hachage (à utiliser)

- Fonction à SENS UNIQUE
- motdepasse -> \$2b\$10\$Xy7...
- Impossible de revenir en arrière
- On COMPARE, on ne déchiffre pas
- bcrypt, argon2, scrypt

À NE PAS faire

- Stocker en clair (jamais !)
- Chiffrer (réversible = risqué)
- MD5 / SHA1 seuls (trop rapides)
- Réinventer son propre algo
- Logguer le mot de passe

Le sel (salt) : bcrypt ajoute une valeur aléatoire à chaque mot de passe avant de le hacher. Résultat : deux personnes avec le même mot de passe ont des hash différents, et les attaques par table précalculée (rainbow tables) tombent à l'eau.

bcrypt en pratique

Hacher à l'inscription, comparer à la connexion

Installation :

```
npm install bcryptjs && npm install -D @types/bcryptjs
```

bash

```
import bcrypt from 'bcryptjs';

// 1) A L'INSCRIPTION : hacher (10 = cout / nb de tours)
const hash = await bcrypt.hash(motDePasse, 10);
// -> '$2b$10$N9qo8uLOickgx2ZMRZoMy...' : c'est CA qu'on stocke

// 2) A LA CONNEXION : comparer (sans jamais dechiffrer)
const ok = await bcrypt.compare(motDePasseEnvoye, hash);
if (!ok) { /* mauvais mot de passe -> 401 */ }
```

TS

On utilise bcryptjs (100% JavaScript) : aucune compilation native, parfait sous Windows. bcrypt.compare re-hache et compare : on ne « décode » jamais le hash.

PARTIE 4

Le JSON Web Token en détail

Anatomie d'un JWT

Trois parties séparées par des points



```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 . eyJzdWIiOiJEsInJvbGUiOiJVU0VSIn0 . Sf1Kxw_RJSMekKF2QT4f...
```

- **Header & Payload** = juste du JSON encodé en base64url (PAS chiffré)
- **N'importe qui peut LIRE le payload** -> jamais de mot de passe ni de secret dedans
- **La Signature** est calculée avec un SECRET connu du seul serveur
- **Modifier le payload casse la signature** -> le serveur rejette le token

Signé n'est pas chiffré

Le point le plus mal compris du JWT

Ce que la signature GARANTIT

- Intégrité : le contenu n'a pas changé
- Authenticité : émis par notre serveur
- Sans le secret, impossible de forger
- un token valide

Ce qu'elle ne fait PAS

- Elle ne cache PAS le contenu
- Le payload est lisible par tous
- (coller le token sur jwt.io le prouve)
- Donc : aucune donnée sensible dedans

Que mettre dans le payload (les « claims ») ?

- **OK** : l'id de l'utilisateur (sub), son rôle, la date d'expiration (exp)
- **PAS OK** : mot de passe, numéro de carte, infos confidentielles
- **Règle** : juste de quoi identifier l'utilisateur et ses droits

jsonwebtoken : signer et vérifier

Deux fonctions : `jwt.sign` et `jwt.verify`

Installation :

```
npm install jsonwebtoken  &&  npm install -D @types/jsonwebtoken
```

bash

```
import jwt from 'jsonwebtoken';
const SECRET = process.env.JWT_SECRET!;  // dans .env, jamais dans le code

// SIGNER (au login) : creer le token
const token = jwt.sign(
  { sub: user.id, role: user.role },  // payload
  SECRET,
  { expiresIn: '1h' }                 // expiration
);

// VERIFIER (sur une route protegee) : lance une erreur si invalide/expire
const payload = jwt.verify(token, SECRET);  // { sub, role, iat, exp }
```

TS

Dans `.env` : `JWT_SECRET="une_longue_chaine_aleatoire_secrete"`. Changez-la en prod, et ne la committez jamais (elle est dans `.gitignore`).

PARTIE 5

Implémentation dans Express

Le modèle User (Prisma)

Un champ password (haché) et un rôle (enum)

```
enum Role {  
  USER  
  ADMIN  
}  
  
model User {  
  id      Int      @id @default(autoincrement())  
  email   String    @unique  
  password String    // le HASH bcrypt  
  role    Role      @default(USER)  
  createdAt DateTime @default(now())  
}
```

prisma

Points clés

- email @unique : 1 compte / email
-
- password stocke le HASH,
- jamais le mot de passe en clair
-
- role : enum USER / ADMIN
- pour l'autorisation
-
- Après edit du schema :
- npx prisma migrate dev

POST /auth/register

Créer un compte : on hache AVANT d'enregistrer

```
import { Router } from 'express';
import bcrypt from 'bcryptjs';
import prisma from '../utils/prisma';
```

```
const routerAuth = Router();
```

```
routerAuth.post('/register', async (req, res) => {
  const { email, password } = req.body;
  const hash = await bcrypt.hash(password, 10); // on hache
  try {
    const user = await prisma.user.create({ data: { email, password: hash } });
    res.status(201).json({ id: user.id, email: user.email }); // jamais le hash !
  } catch (e) {
    res.status(400).json({ erreur: 'Email déjà utilise' });
  }
});
```

TS

→ User
→ prisma.user.findUnique({where: {email}})
→ const ok = bcrypt.compare(password, user.password) (RSA)
const ok = bcrypt.compare(password, user.password) (RSA)
const token = jwt.sign({sub: user.id, role: user.role},
 process.env.JWT_SECRET!,
 {expiresIn: '1h'});

POST /auth/login

Vérifier le mot de passe, puis émettre un JWT

```
import jwt from 'jsonwebtoken';

routerAuth.post('/login', async (req, res) => {
  const { email, password } = req.body;
  const user = await prisma.user.findUnique({ where: { email } });

  // message identique si email OU mot de passe faux (ne pas en dire trop)
  if (!user) return res.status(401).json({ erreur: 'Identifiants invalides' });
  const ok = await bcrypt.compare(password, user.password);
  if (!ok) return res.status(401).json({ erreur: 'Identifiants invalides' });

  const token = jwt.sign({ sub: user.id, role: user.role },
    process.env.JWT_SECRET!, { expiresIn: '1h' });
  res.json({ token });
});
```

TS

xxx.yyy.zzz

Le middleware d'authentification

Lire le header Authorization et vérifier le token

```
// src/middlewares/auth.ts
import { Request, Response, NextFunction } from 'express';
import jwt from 'jsonwebtoken';

export function authentifier(req: Request, res: Response, next: NextFunction) {
  const header = req.headers.authorization; // 'Bearer xxx.yyy.zzz'
  if (!header?.startsWith('Bearer ')) {
    return res.status(401).json({ erreur: 'Token manquant' });
  }
  const token = header.split(' ')[1];
  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET!);
    (req as any).user = payload; // on attache l'utilisateur
    next(); // -> la route s'exécute
  } catch {
    res.status(401).json({ erreur: 'Token invalide ou expire' });
  }
}
```

TS

Comment ça marche

- Le client envoie :
- Authorization:
- Bearer <token>
-
- On extrait le token
- On le vérifie (verify)
- On attache req.user
- On appelle next()
-
- Sinon -> 401
- (non authentifié)

Protéger une route

Brancher le middleware avant le handler

```
import { authentifier } from './middlewares/auth';

// Route PUBLIQUE : tout le monde peut lire le Pokedex
app.get('/pokedex', listerPokedex);

// Route PROTEGEE : il faut un token valide pour capturer
app.post('/pokedex/capturer/:nom', authentifier, capturer);

// Dans le handler, on connaît l'utilisateur :
function capturer(req, res) {
  const userId = (req as any).user.sub; // vient du token
  // ...
}
```

(req: Request)

TS

On place *authentifier* ENTRE le chemin et le handler. Express l'exécute d'abord : si le token est valide, *next()* laisse passer ; sinon, la réponse 401 est déjà envoyée et le handler n'est jamais atteint. On peut protéger tout un router avec *router.use(authentifier)*.

Autorisation par rôle (RBAC)

401 = pas authentifié, 403 = pas autorisé

```
// middleware qui exige un role precis
export function exigerRole(role: string) {
  return (req, res, next) => {
    if ((req as any).user.role !== role) {
      return res.status(403).json({ erreur: 'Acces refuse' });
    }
    next();
  };
}

// seul un ADMIN peut supprimer
app.delete('/pokedex/:id',
  authentifier, exigerRole('ADMIN'), relacher);
```

TS

401 vs 403

- 401 Unauthorized :
- « je ne sais pas qui tu es »
- (token absent / invalide)
-
- 403 Forbidden :
- « je sais qui tu es, mais
- tu n'as pas le droit »
- (mauvais rôle)

On chaîne les middlewares : authentifier (qui es-tu ?) PUIS exigerRole (as-tu le droit ?). L'ordre compte : on identifie avant d'autoriser.

Le flux complet, de bout en bout

Register -> Login -> Requête protégée



```
### 1. Creer un compte
POST http://localhost:3000/auth/register
Content-Type: application/json
{ "email": "alice@cmaisonneuve.qc.ca", "password": "secret123" }

### 2. Se connecter -> recupere { "token": "..." }
POST http://localhost:3000/auth/login
Content-Type: application/json
{ "email": "alice@cmaisonneuve.qc.ca", "password": "secret123" }

### 3. Appeler une route protegee avec le token
POST http://localhost:3000/pokedex/capturer/pikachu
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

http

PARTIE 6

Sécurité & bonnes pratiques

Bonnes pratiques de sécurité

Le minimum vital à respecter

Mots de passe

- Toujours bcrypt (jamais en clair)
- Ne jamais renvoyer le hash
- Exiger une longueur minimale
- Message de login générique

Tokens

- Secret long, dans .env
- Expiration courte (expiresIn)
- HTTPS obligatoire en prod
- Aucune donnée sensible dedans

Côté serveur

- Valider les entrées (Zod)
- cors + helmet
- Limiter les tentatives (rate limit)
- Ne pas logger secrets / tokens

Pour aller plus loin (hors séance) :

- **Refresh tokens** : un token court + un token long pour se reconnecter sans login
- **Stockage côté client** : cookie httpOnly (plus sûr) vs localStorage (simple, sensible au XSS)
- **Solutions clés en main** : Clerk, Auth0, Lucia... quand on ne veut pas tout coder soi-même

Erreurs fréquentes à éviter

Les pièges qu'on voit le plus souvent

Erreur	Conséquence	Correctif
Mot de passe en clair	Fuite catastrophique	<code>bcrypt.hash</code>
Secret JWT dans le code	Tokens forgeables	<code>process.env.JWT_SECRET</code>
Données sensibles dans le payload	Lisible par tous	Seulement id + rôle
Pas d'expiration	Token valide à vie	<code>expiresIn: '1h'</code>
Renvoyer <code>user.password</code>	Hash exposé	Sélectionner les champs
401 et 403 confondus	Mauvais diagnostic	401 authN, 403 authZ
Vérifier le token à la main	Bugs / failles	<code>jwt.verify</code>

Récapitulatif de la séance

Ce qu'on a construit aujourd'hui

Concepts

- authN vs authZ
- HTTP stateless
- sessions vs tokens
- signé != chiffré
- rôles (RBAC)

Outils

- bcryptjs : hash + compare
- jsonwebtoken : sign + verify
- JWT_SECRET dans .env
- Role enum (Prisma)
- header Authorization

Routes & middlewares

- POST /auth/register
- POST /auth/login
- authentifier (401)
- exigerRole (403)
- routes publiques / protégées

Les commandes à mémoriser :

```
npm install bcryptjs jsonwebtoken
npm install -D @types/bcryptjs @types/jsonwebtoken
# .env : JWT_SECRET="une_longue_chaine_secrete"
```

bash

Pour la prochaine séance

On consolide et on prépare la suite

À faire de votre côté

- Ajouter le modèle User (password, role)
- Coder /auth/register et /auth/login
- Écrire le middleware authentifier
- Protéger capturer (token requis)
- Réserver DELETE aux ADMIN
- Tester le flux complet dans Postman

Au programme ensuite

- Refresh tokens & déconnexion
- Validation des entrées (Zod)
- Frontend React + login
- Stocker le token côté client
- Envoyer le token avec axios
- Déploiement (Vercel / Render)

Votre API sait maintenant QUI parle et CE QU'IL a le droit de faire.

C'est la dernière brique backend avant de brancher un vrai frontend.



Questions ?

Prochaine étape : sécuriser le Pokédex avec register / login / JWT.

Hachez, signez, vérifiez. Et ne committez jamais votre secret.

